

Sesión 9: Despliegue de modelos (2) — De API a servicio con Docker

Duración: 2,5 horas | Modalidad: Virtual síncrona

Objetivo	Empaquetar la API de FastAPI en un contenedor Docker listo para desplegar en cualquier entorno, y orquestar el stack completo con Docker Compose.
Herramientas	Docker, Docker Compose, Uvicorn, FastAPI, MLflow
Prerrequisitos	API de FastAPI funcionando localmente (sesión 8). Docker Desktop instalado.

BLOQUE 1 — Dockerfile para la API de FastAPI (30 min)

Un Dockerfile para una API web tiene algunas diferencias importantes respecto al de un script de entrenamiento. Lo más crítico: el servidor debe ser accesible desde fuera del contenedor.

Fichero: Dockerfile

```
# — STAGE 1: imagen base —————
# python:3.10-slim reduce el tamaño ~70% vs python:3.10
FROM python:3.10-slim

# Evitar que Python escriba .pyc y que el output quede en buffer
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

# — STAGE 2: dependencias —————
# Copiar requirements ANTES que el código fuente
# Así Docker cachea esta capa y no reinstala si solo cambia el código
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# — STAGE 3: código fuente —————
COPY . .

# Puerto que expone la aplicación (documentación, no abre el puerto)
EXPOSE 8000
```

```
# — STAGE 4: comando de arranque —————
# CRÍTICO: --host 0.0.0.0 hace que el servidor escuche en todas las
# interfaces, no solo en localhost. Sin esto, el contenedor no es
# accesible desde fuera.
CMD ["uvicorn", "main:app",
     "--host", "0.0.0.0",
     "--port", "8000",
     "--workers", "1"]
```

Fichero: .dockerignore

```
# Excluir lo que no necesita el contenedor
__pycache__/
*.py[cod]
.git/
.gitignore
*.md
tests/
.env
*.ipynb
mlruns/           # No incluir los experimentos locales
```

Construir y probar la imagen:

```
# Construir la imagen (etiquetada como 'sentiment-api' versión 1.0)
docker build -t sentiment-api:1.0 .

# Ver el tamaño de la imagen construida
docker images sentiment-api

# Ejecutar el contenedor
# -p 8000:8000 → mapea puerto host:contenedor
# --rm        → elimina el contenedor al parar
# -e          → variables de entorno
docker run --rm -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://host.docker.internal:5000 \
  -e MODEL_NAME=SentimentAnalyzer \
  -e MODEL_STAGE=Production \
  sentiment-api:1.0

# Verificar desde el host (otra terminal)
curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict \
  -H 'Content-Type: application/json' \
  -d '{"text": "Fantastic product!"}'
```

host.docker.internal

Dentro de un contenedor Docker, 'localhost' se refiere al propio contenedor, no a tu máquina. Para acceder a servicios del host (como MLflow corriendo en tu máquina), usa

host.docker.internal (funciona en Docker Desktop para Mac y Windows). En Linux, usa la IP de la interfaz docker0 o usa --network=host.

BLOQUE 2 — Estrategia: modelo empaquetado dentro de la imagen (30 min)

En lugar de conectar al servidor MLflow en tiempo de ejecución, podemos descargar el modelo durante el build de la imagen. Esto tiene ventajas e inconvenientes:

Modelo empaquetado en imagen	Modelo cargado desde MLflow en runtime
No depende de MLflow en runtime	Imagen ligera, modelo siempre actualizado
Imagen pesada (modelo incluido)	Requiere MLflow accesible en producción

Dockerfile con modelo empaquetado (alternativa)

```
FROM python:3.10-slim

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Descargar el modelo de HuggingFace durante el build
# Así no necesitamos conexión a MLflow en runtime
RUN python -c "
from transformers import pipeline; \
pipe = pipeline('sentiment-analysis', \
    model='distilbert-base-uncased-finetuned-sst-2-english'); \
pipe.save_pretrained('/app/model')"

COPY . .
EXPOSE 8000

# Variable de entorno para que main.py use el modelo local
ENV MODEL_PATH=/app/model
ENV USE_LOCAL_MODEL=true

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

BLOQUE 3 — Docker Compose: el stack completo (40 min)

En producción raramente tenemos un solo contenedor. Nuestro stack completo incluye: la API de FastAPI y el servidor de MLflow (para cargar modelos y registrar nuevas versiones).

Fichero: docker-compose.yml

```
version: '3.8'

services:

  # — Servidor de MLflow —————
  mlflow:
    image: python:3.10-slim
    command: >
      sh -c 'pip install mlflow &&
        mlflow server
        --host 0.0.0.0
        --port 5000
        --backend-store-uri /mlflow/mlruns
        --default-artifact-root /mlflow/artifacts'
    ports:
      - '5000:5000'
    volumes:
      - mlflow_data:/mlflow
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:5000/health']
      interval: 10s
      timeout: 5s
      retries: 5

  # — API de FastAPI —————
  api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - '8000:8000'
    environment:
      # Dentro de Docker Compose, los servicios se llaman por nombre
      - MLFLOW_TRACKING_URI=http://mlflow:5000
      - MODEL_NAME=SentimentAnalyzer
      - MODEL_STAGE=Production
    depends_on:
      mlflow:
        condition: service_healthy # Espera a que MLflow pase el healthcheck
    restart: unless-stopped

volumes:
  mlflow_data: # Persiste los datos de MLflow entre reinicios
```

Comandos para trabajar con Docker Compose:

```
# Arrancar todo el stack (en background)
```

```

docker compose up -d

# Ver logs de todos los servicios
docker compose logs -f

# Ver logs solo de la API
docker compose logs -f api

# Estado de los servicios
docker compose ps

# Reconstruir la imagen de la API (tras cambios en el código)
docker compose up -d --build api

# Parar y eliminar contenedores (conserva los volúmenes)
docker compose down

# Parar y eliminar TODO incluyendo volúmenes (;borra los datos de MLflow!)
docker compose down -v

# Probar que el stack completo funciona
curl http://localhost:8000/health
curl http://localhost:5000/health # MLflow UI

```

Script de integración completo: test_stack.py

```

# test_stack.py - verifica que el stack completo funciona
import requests
import time

API_URL = 'http://localhost:8000'
MLFLOW_URL = 'http://localhost:5000'

def wait_for_service(url, service_name, max_retries=30):
    for i in range(max_retries):
        try:
            r = requests.get(f'{url}/health', timeout=2)
            if r.status_code == 200:
                print(f'{service_name} listo!')
                return True
        except:
            pass
        print(f'Esperando a {service_name}... ({i+1}/{max_retries})')
        time.sleep(2)
    return False

if __name__ == '__main__':
    print('=== Test del stack completo ===')

    # Esperar a que los servicios arranquen
    assert wait_for_service(MLFLOW_URL, 'MLflow')

```

```

assert wait_for_service(API_URL, 'API')

# Tests de la API
tests = [
    ('This is amazing!', 'POSITIVE'),
    ('I hate this product.', 'NEGATIVE'),
    ('Not sure about this.', None), # Puede ser cualquiera
]

print('\n--- Tests de predicción ---')
for text, expected in tests:
    r = requests.post(f'{API_URL}/predict', json={'text': text})
    data = r.json()
    check = '✓' if (expected is None or data['label'] == expected) else 'X'
    print(f'{check} "{text[:35]}..." → {data["label"]}'
          f'({data["latency_ms"]}ms)')

print('\nStack funcionando correctamente.')

```

TRABAJO AUTÓNOMO (2,5 h)

Tarea para casa

Crear el Dockerfile y docker-compose.yml para el proyecto. Construir la imagen, arrancar el stack con docker compose up y verificar que el endpoint /predict responde correctamente desde fuera del contenedor usando test_stack.py. Este stack contenerizado es el artefacto central del proyecto final.